Validation & Exception Handling Cheat Sheet

1. Layer Responsibilities

Layer / Actor	Responsibility	Validation?	Exception Behavior
Web Framework (Infrastructure)	Extracts HTTP arguments and forwards them to the controller as basic Python types (`str`, `int`, `None`). Displays presenter results.	No domain or application validation.	Does not handle application exceptions; delegates to the controller.
Controller (Interface)	Receives primitive arguments, constructs the DTO, invokes the use case, and returns results to the presenter.	No actual validation. Delegates argument validation to the DTO.	**Catches** `ValueError` from DTO construction/conversion and presents a `VALIDATION_ERROR`.
DTO (Application)	Stores incoming primitive arguments and converts them into Domain types.	**YES — validates incoming arguments.**	**Raises** `ValueError` for invalid arguments. Does not raise business-rule exceptions.
Use Case (Application)	Orchestrates interactions between Domain Entities, repositories, and service ports.	Does not perform basic input validation. Coordinates business operations.	**Catches** exceptions from execution dependencies when appropriate. Does not raise exceptions merely to communicate expected validation failures.
Domain Entities (Domain)	Enforce invariants and protect the integrity of domain objects.	**YES — validates domain invariants and business rules.**	**Raises** domain-specific exceptions when business rules or invariants are violated.
Repositories / Service Ports	Provide persistence and external service operations.	Validate their own operational requirements where appropriate.	May **raise** `ValueError` or domain/business-rule exceptions when operations fail or constraints are violated.

2. Validation Cheat Sheet

Type of Validation	Where It Happens	Example
HTTP argument extraction	Web Framework	Extract `title` from POST body.
Primitive argument validation	DTO	Ensure `title` is a non-empty string.
Type / format conversion	DTO	Convert a string into a `Deadline` or `Priority`.
Domain invariant validation	Domain Entity / Value Object	Ensure a deadline is valid or a priority is an allowed value.
Business-rule validation	Domain	Ensure a completed task cannot be modified, if that is a business rule.
Infrastructure / operational validation	Repository / Service	Ensure a requested resource exists or an external operation can proceed.

Core principle: Validate each thing at the layer that owns the rule. Don't duplicate validation across templates, controllers, DTOs, and domain entities.

3. Exception Handling Cheat Sheet

DTO (Application Layer)

- **Raises:** `ValueError`.
- Used for invalid incoming arguments, malformed data, and failed primitive-to-domain conversions.
- Does NOT raise business-rule exceptions.

Controller (Interface Layer)

- **Catches:** `ValueError`.
- Converts DTO validation failures into a `VALIDATION_ERROR` presenter response.
- Should not perform the validation itself.
- Should not allow expected input-validation errors to leak into the web framework.

Use Case (Application Layer)

- Coordinates the execution of the requested operation.
- May catch exceptions from repositories, service ports, and domain operations.
- Should handle exceptions when it has enough context to make a meaningful application-level decision.
- Should not indiscriminately catch every exception.

Domain (Domain Layer)

- **Raises:** Domain-specific exceptions.
- Protects business rules and entity invariants.
- Should not depend on HTTP, framework, or presentation-layer exceptions.

Repositories / Service Ports

- **May raise:** `ValueError`, domain-specific exceptions, and infrastructure-specific exceptions.
- Report failures to the layer coordinating their operation.
- Should not silently swallow failures.

4. The Typical Request Flow

1. **Web Framework:** Extracts primitive arguments from the HTTP request.
2. **Controller:** Passes those arguments into the DTO.
3. **DTO:** Validates arguments and converts them into appropriate Domain types.
4. **Controller:** Passes the DTO to the Use Case.
5. **Use Case:** Calls `to_execution_params()` to obtain the arguments needed for execution.
6. **Use Case:** Coordinates Domain Entities, repositories, and service ports.
7. **Domain / Infrastructure:** Enforces business rules and performs operations, raising exceptions when necessary.
8. **Use Case / Controller:** Handles exceptions at the appropriate boundary.
9. **Presenter / Web Framework:** Communicates the outcome to the user.

5. The Golden Rules

- **Web Framework:** Extract and forward; don't own application validation.
- **Controller:** Translate incoming requests into application calls; catch and present expected input-validation failures.
- **DTO:** Validate primitive inputs and convert them into Domain types; raise `ValueError`.
- **Use Case:** Orchestrate business operations and handle failures when it can meaningfully respond to them.
- **Domain:** Own business rules and invariants; raise domain-specific exceptions.
- **Repositories / Service Ports:** Perform their operations and report failures through exceptions.

Remember

- Validation belongs to the layer that owns the rule.

- Raising an exception and catching an exception are different responsibilities.
- Catch exceptions at the layer that knows what to do with them.
- Don't catch exceptions just to hide them or convert everything into a generic error.
- Don't use `ValueError` for every failure. Use domain-specific exceptions for business-rule violations.

One Important Architectural Nuance

Your original notes say the use case catches exceptions but does not raise them. I would avoid treating that as an absolute rule. A use case can legitimately raise an application-specific exception when it needs to communicate an application-level failure. The more useful principle is that exceptions should be handled at the boundary that knows how to respond to them.

Likewise, repositories and service ports shouldn't automatically raise `ValueError` for every failure. Infrastructure failures, missing resources, and business-rule violations are different kinds of problems and may deserve different exception types.